



UNS
UNIVERSIDAD
NACIONAL DEL SANTA

ARQUITECTURA ANDROID

CURSO: **APLICACIONES MÓVILES**



Dr. MIRKO MANRIQUE RONCEROS



OBJETIVO DE LA SESIÓN

Comprender la arquitectura del sistema operativo Android, identificando sus principales capas y componentes, así como el rol de las librerías y del entorno de desarrollo, para aplicar estos conocimientos en la creación de aplicaciones móviles eficientes y estructuradas

1 Comprender la arquitectura de Android



Identificar las principales capas y componentes del sistema
Entender cómo interactúan las diferentes capas
Reconocer el rol de cada componente

2 Identificar el rol de las librerías



Conocer librerías del Android SDK
Entender librerías de Google y de terceros
Aplicar librerías en el desarrollo

3 Entender el entorno de aplicación



Procesos, memoria y ciclo de vida
Gestión de permisos y seguridad
Comunicación entre procesos

4 Aplicar conocimientos prácticos



Crear aplicaciones móviles eficientes
Estructurar código de manera modular
Optimizar recursos del dispositivo



PREGUNTAS MOTIVACIONALES



Pregunta motivacional

Reflexiona sobre la importancia de la arquitectura en el desarrollo de software



¿Si comparamos la construcción de una app con la de un edificio, ¿qué creen que pasaría si el electricista, el gasfitero y el albañil trabajaran todos sobre la misma pared al mismo tiempo sin un plano?

Piensa en las consecuencias de trabajar sin una estructura definida...



Para reflexionar

La arquitectura de software es como el plano de construcción que define cómo se organizan los componentes de una aplicación.



PREGUNTAS MOTIVACIONALES



¿Construir sin plano?

¿Si comparamos la construcción de una app con la de un edificio, ¿qué creen que pasaría si el electricista, el gasfitero y el albañil trabajaran todos sobre la misma pared al mismo tiempo sin un plano?



Problema

Se estorbarían

Habría accidentes

Sin saber a quién culpar



Consecuencia

No saber cómo arreglarlo

Tumbar la pared entera

Código espagueti



Solución

Arquitectura como plano

Define dónde va cada cosa

Antes de poner el primer ladrillo



Idea clave

Se estorbarían, habría accidentes y, si algo falla, no sabríamos a quién culpar ni cómo arreglarlo sin tumbar la pared entera. En software, esto es el "código espagueti". La arquitectura es ese plano de construcción que define dónde va cada cosa antes de poner el primer ladrillo (o línea de código).



PREGUNTAS MOTIVACIONALES



¿Pérdida de datos al cambiar de estado?

¿Alguna vez han usado una app que, al girar la pantalla o recibir una llamada, se cierra o pierde todo lo que habían escrito?



Rotación de pantalla

Al girar el dispositivo, la Activity se recrea y puede perder el estado actual



Llamada entrante

Recibir una llamada puede cerrar la app o perder los datos ingresados



Problema común

Esta experiencia frustrante ocurre en muchas apps mal diseñadas



Pregunta clave

¿Por qué creen que sucede esto? Piensen en qué puede estar faltando en la arquitectura de la app...



PREGUNTAS MOTIVACIONALES



¿Alguna vez han usado una app que, al girar la pantalla o recibir una llamada, se cierra o pierde todo lo que habían escrito?

¿Por qué creen que sucede esto?



El problema

La app no tiene estructura

No separa UI de memoria

Pierde estado al cambiar



La causa

Falta de arquitectura

Estado no persistido

Memoria no gestionada



La solución

Arquitectura robusta

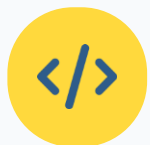
Separación de responsabilidades

Apps resilientes

Respuesta clave



Porque la app no tiene una estructura que separe la apariencia de la memoria. Al no haber arquitectura, la app "olvida" lo que estaba haciendo al cambiar de estado. Hoy aprenderemos que una buena arquitectura hace que una app sea "resiliente" a estos cambios.



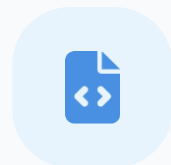
¿Un solo archivo para todo?

Imaginemos escenarios de arquitectura de software

¿Imaginen que **Facebook** o **WhatsApp** fueran una sola lista gigante de código (un solo archivo). Si un programador quisiera cambiar el color de un botón y se equivoca, ¿podría colapsar todo el sistema de mensajes?



Piensa en las consecuencias de un código monolítico vs modular



Código monolítico



Arquitectura modular



Sistema robusto

? PREGUNTAS MOTIVACIONALES



Pregunta motivacional

Reflexiona sobre los riesgos de trabajar sin arquitectura adecuada



¿Imaginen que Facebook o WhatsApp fueran una sola lista gigante de código (un solo archivo). Si un programador quisiera cambiar el color de un botón y se equivoca, ¿podría colapsar todo el sistema de mensajes?

Piensa en las consecuencias de tener todo el código en un solo lugar...



Respuesta

¡Absolutamente! Sin arquitectura, todo está pegado con pegamento fuerte. El objetivo de hoy es aprender a construir apps como si fueran piezas de LEGO: donde podemos cambiar una pieza dañada por una nueva sin que el resto del castillo se caiga.



PREGUNTAS MOTIVACIONALES



Pregunta motivacional

Reflexiona sobre el tamaño y rendimiento de las aplicaciones móviles



¿Por qué creen que algunas aplicaciones pesan apenas 20MB y son potentes, mientras otras pesan 500MB y son lentas? ¿Es solo culpa del internet?

Piensa en la relación entre el tamaño de la app y su eficiencia...



Para reflexionar

El tamaño y rendimiento de una app dependen de cómo está organizada internamente, no solo del contenido.



PREGUNTAS MOTIVACIONALES



Pregunta motivacional

Reflexiona sobre la eficiencia y el peso de las aplicaciones



¿Por qué creen que algunas aplicaciones pesan apenas 20MB y son potentes, mientras otras pesan 500MB y son lentas? ¿Es solo culpa del internet?



Respuesta

No siempre. Muchas veces es el desorden interno. Una buena arquitectura permite que la app use solo los recursos que necesita en el momento justo. Aprender arquitectura es aprender a ser eficientes con los recursos del usuario (batería, RAM y datos).



Para reflexionar

La arquitectura de software impacta directamente en el rendimiento y el consumo de recursos de las aplicaciones móviles.



PREGUNTAS MOTIVACIONALES



Pregunta motivacional

Reflexiona sobre la importancia de la arquitectura escalable y mantenible



Si hoy creas una app exitosa y mañana te contrata una empresa grande para mejorarla, ¿qué preferirías entregarle: un código que solo tú entiendes o un sistema que cualquier ingeniero del mundo pueda seguir?

Piensa en la escalabilidad y mantenibilidad a largo plazo de tu código...



Para reflexionar

La arquitectura clara facilita que otros desarrolladores puedan entender y continuar el trabajo en tu aplicación.



PREGUNTAS MOTIVACIONALES



Pregunta motivacional

Reflexiona sobre la importancia de escribir código mantenible y escalable



Si hoy creas una app exitosa y mañana te contrata una empresa grande para mejorarla, ¿qué preferirías entregarle: un código que solo tú entiendes o un sistema que cualquier ingeniero del mundo pueda seguir?



Respuesta

Un sistema estándar. La arquitectura es el lenguaje universal de los programadores senior. Si quieres trabajar en Google, Amazon o una gran Startup, ellos no buscan que sepas programar, buscan que sepas construir sistemas que otros puedan continuar.

Contexto y relevancia de Android en el desarrollo móvil

En la actualidad, el sistema operativo Android se ha consolidado como una de las plataformas más utilizadas en el desarrollo de aplicaciones móviles a nivel mundial. Su popularidad se debe no solo a su carácter abierto y flexible, sino también a la robusta arquitectura sobre la cual está construido. Comprender la arquitectura de la plataforma Android es fundamental para cualquier desarrollador, ya que esta define cómo interactúan sus diversos componentes: desde el núcleo del sistema operativo hasta las aplicaciones que el usuario final ejecuta. Cada capa —el núcleo de Linux, las bibliotecas nativas, el entorno de ejecución ART, el marco de aplicaciones y las aplicaciones mismas— cumple un rol esencial en el funcionamiento del sistema.

En este contexto, las librerías de Android desempeñan un papel clave al proporcionar funcionalidades reutilizables que optimizan el proceso de desarrollo. Existen tanto librerías oficiales como de terceros, orientadas a aspectos diversos como la interfaz gráfica, el acceso a redes, la persistencia de datos, la gestión de imágenes, la animación o la seguridad. El uso eficaz de estas librerías permite crear aplicaciones más robustas, eficientes y con mejores experiencias de usuario, al mismo tiempo que se reduce significativamente el tiempo de desarrollo.

Importancia de la arquitectura

Una sólida comprensión de la arquitectura Android permite a los desarrolladores crear aplicaciones más eficientes, mantenibles y escalables, optimizando el rendimiento y la experiencia del usuario final.



I. ARQUITECTURA DE LA PLATAFORMA ANDROID

Estructura y componentes de la arquitectura Android

La arquitectura de Android es una pila de componentes de software que se organiza en varias capas. Cada capa se comunica con la capa inferior y proporciona servicios a la capa superior. Esta arquitectura modular permite una gran flexibilidad, reutilización de componentes y portabilidad a diferentes dispositivos.



Aplicaciones (Capa superior)

Aplicaciones del sistema y de terceros instaladas por el usuario



Framework de Aplicaciones

APIs de alto nivel para desarrollo de aplicaciones



Android Runtime (ART)

Entorno de ejecución para aplicaciones Android



Librerías Nativas C/C++

Bibliotecas nativas para funcionalidades esenciales



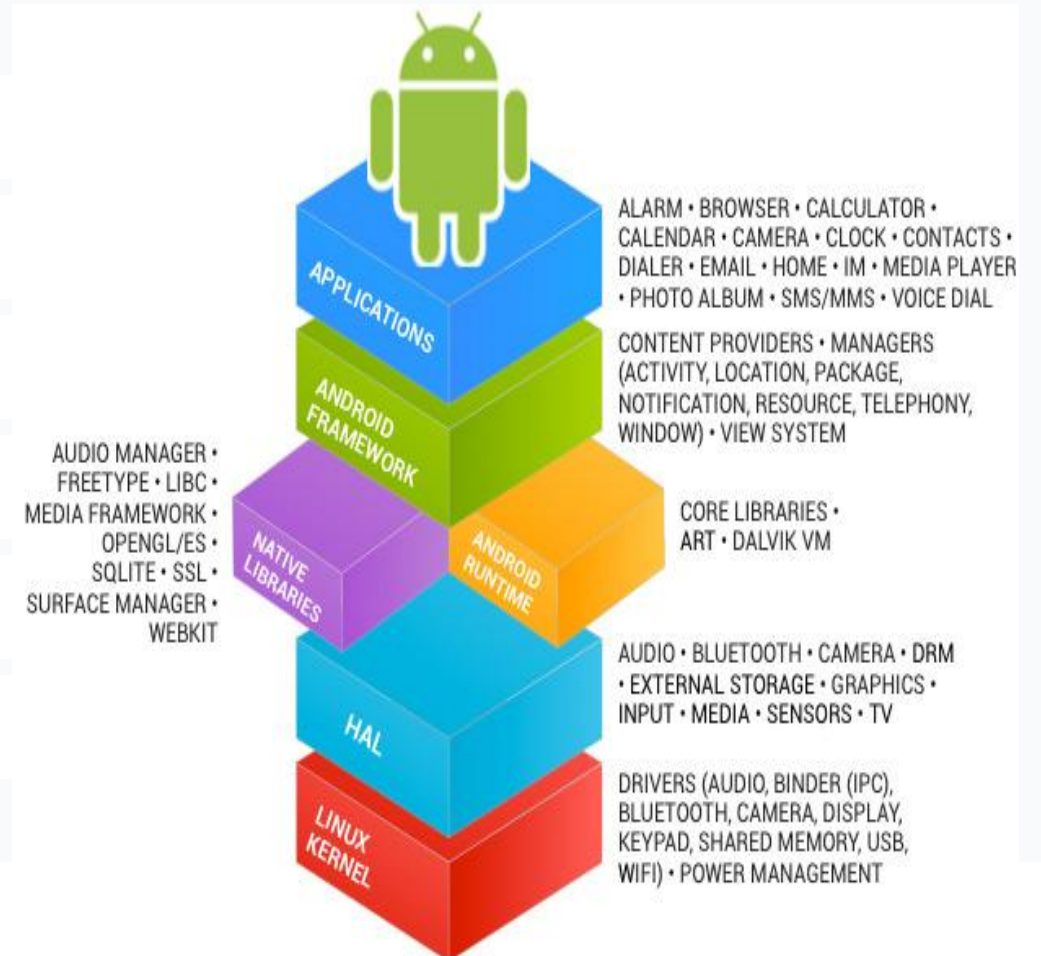
HAL (Hardware Abstraction Layer)

Interfaz estándar entre framework y hardware



Kernel de Linux (Capa base)

Núcleo del sistema operativo Android





CAPA DE APLICACIONES

Aplicaciones construidas sobre el Framework

Esta es la capa superior donde se ejecutan todas las aplicaciones del usuario



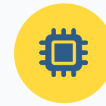
Construidas sobre Framework

Base sólida de Android

Desarrolladas utilizando el Framework de Aplicaciones

Acceso a APIs de alto nivel

Componentes reutilizables (Activities, Services)



Ejecutan en ART

Android Runtime

Se ejecutan dentro del entorno ART

Optimización de bytecode DEX

Gestión de memoria y rendimiento



Diversidad de Apps

Productividad, juegos, social

Productividad: Gmail, Calendar, Drive

Juegos: Categorías variadas

Social: WhatsApp, Facebook, Twitter



Acceso a Capas Inferiores

Vía APIs seguras

Acceso a hardware a través del HAL

Almacenamiento a través del kernel

Comunicación entre procesos



La base fundamental del sistema Android

En la base de la arquitectura de Android se encuentra el kernel de Linux. Android no es simplemente una distribución de Linux tradicional, pero utiliza el kernel de Linux como su núcleo. Google ha realizado modificaciones significativas en el kernel de Linux para adaptarlo a las necesidades de un sistema operativo móvil, como la gestión de energía, la administración de memoria, la seguridad y el soporte para hardware específico de dispositivos móviles.



Gestión de Memoria

Administración eficiente de la memoria RAM y asignación de recursos



Gestión de Energía

Optimización del consumo de batería en dispositivos móviles



Seguridad

Permisos y aislamiento de procesos



Gestión de Redes

Soporte para Wi-Fi, Bluetooth y datos móviles



Importancia del Kernel

El kernel de Linux es el núcleo del sistema operativo Android, proporcionando la base para la gestión de procesos, memoria, redes y hardware del dispositivo.



Características y funciones fundamentales del kernel de Linux en Android



1. Gestión de Procesos

El kernel de Linux es responsable de la creación, planificación y gestión de los procesos que se ejecutan en el dispositivo.



3. Gestión de Redes

Proporciona la pila de protocolos de red (TCP/IP) y gestiona las conexiones (Wi-Fi, Bluetooth, datos móviles).



5. Controladores de Dispositivos (Drivers)

Contiene los drivers que permiten al SO interactuar con el hardware (pantalla táctil, cámara, sensores, etc.). Los fabricantes contribuyen con drivers específicos.



7. Gestión de Energía

Optimiza el uso de la batería mediante la suspensión de procesos inactivos y la gestión inteligente de la frecuencia del CPU.



2. Gestión de Memoria

Administra la asignación y liberación de memoria. Android utiliza mecanismos como el **"low memory killer"** para liberar memoria bajo presión.



4. Sistema de Archivos

Soporta varios sistemas de archivos, como EXT4, y gestiona el acceso al almacenamiento del dispositivo.



6. Seguridad

Implementa mecanismos de seguridad a nivel del sistema, como permisos y aislamiento de procesos para proteger el entorno.



8. Binder IPC (Inter-Process Communication)

Mecanismo crucial para la comunicación eficiente entre los diferentes procesos y servicios que se ejecutan en Android.



Funciones principales del kernel de Linux en Android

1

Gestión de Procesos

El kernel de Linux es responsable de la creación, planificación y gestión de los procesos que se ejecutan en el dispositivo.

2

Gestión de Memoria

Administra la asignación y liberación de memoria. Android utiliza el "low memory killer" para liberar memoria bajo presión.

3

Gestión de Redes

Proporciona la pila de protocolos de red (TCP/IP) y gestiona conexiones Wi-Fi, Bluetooth y datos móviles.

4

Sistema de Archivos

Soporta varios sistemas de archivos como EXT4 y gestiona el acceso al almacenamiento del dispositivo.

5

Controladores de Dispositivos

Contiene drivers que permiten al sistema interactuar con hardware específico (pantalla táctil, cámara, sensores).

6

Seguridad

Implementa mecanismos de seguridad a nivel del sistema, como permisos y aislamiento de procesos.

7

Gestión de Energía

Optimiza el uso de la batería mediante técnicas como suspensión de procesos inactivos.

8

Binder IPC

Mecanismo crucial para la comunicación eficiente entre procesos y servicios en Android.



Ejemplo de interacción entre aplicación y hardware

Ejemplo: La interacción de una aplicación con la cámara del dispositivo pasa a través de las APIs de Android, que a su vez utilizan los drivers de la cámara gestionados por el kernel de Linux): Un mecanismo crucial para la comunicación eficiente entre los diferentes procesos y servicios que se ejecutan en Android

Flujo de interacción con hardware

La aplicación → APIs de Android → Drivers del kernel → Hardware del dispositivo. Este flujo permite que las aplicaciones accedan al hardware de manera controlada y segura, gestionada por el kernel de Linux.

Comunicación entre procesos (IPC)

Binder IPC es el mecanismo que permite la comunicación eficiente entre los diferentes procesos y servicios que se ejecutan en Android, gestionado por el kernel de Linux.



HAL (HARDWARE ABSTRACTION LAYER)

Interfaz estándar entre framework y hardware

La capa HAL actúa como una interfaz estándar entre el framework de Android (la capa superior) y el hardware específico del dispositivo (que está controlado por el kernel de Linux). El HAL define una interfaz abstracta para los componentes de hardware, lo que permite que el framework de Android interactúe con el hardware sin necesidad de conocer los detalles de implementación de cada fabricante.



Función principal de HAL

El HAL proporciona una capa de abstracción que oculta las diferencias entre las implementaciones de hardware de diferentes fabricantes, permitiendo que Android se ejecute en una amplia variedad de dispositivos.



HAL (HARDWARE ABSTRACTION LAYER)

Características y funciones clave del HAL en Android



1. Abstracción de Hardware

Proporciona una capa de abstracción que oculta las diferencias entre las implementaciones de hardware de diferentes fabricantes.



2. Módulos HAL

El HAL se implementa a través de módulos (bibliotecas dinámicas .so) que son cargados por el sistema Android cuando es necesario. Cada módulo implementa una interfaz estándar para un tipo específico de hardware.



3. Interfaces Estándar

Google define interfaces HAL estándar para los componentes de hardware comunes. Los fabricantes implementan estas interfaces para que su hardware sea compatible con Android.



4. Facilita la Portabilidad

Permite que Android se ejecute en una amplia variedad de hardware, ya que los fabricantes solo necesitan proporcionar las implementaciones de los módulos HAL para sus dispositivos.

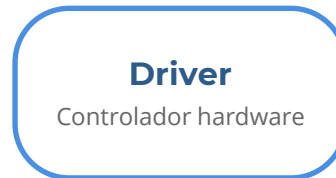


HAL (HARDWARE ABSTRACTION LAYER)



Ejemplo práctico de la HAL

Cuando una aplicación quiere acceder a la cámara, interactúa con las APIs del framework de Android. Estas APIs, a través del HAL de la cámara, se comunican con el driver de la cámara específico del hardware en el kernel de Linux.



Abstracción

El HAL oculta los detalles específicos del hardware, permitiendo que el framework interactúe sin conocer implementaciones.



Portabilidad

Facilita que Android se ejecute en diversos dispositivos con hardware diferente.

Capa de Librerías Nativas

Esta capa contiene un conjunto de bibliotecas nativas escritas en C y C++ que proporcionan funcionalidades esenciales para los componentes de nivel superior de Android. Estas bibliotecas son utilizadas por el framework de Android a través del HAL.

Importancia de las Librerías Nativas

Las librerías nativas proporcionan funcionalidades fundamentales como gráficos 2D y 3D (Skia, OpenGL ES), códecs multimedia (libvpx, libstagefright), bases de datos (SQLite), seguridad (SSL/TLS, OpenSSL), soporte para lenguajes (libc), etc. Al ser escritas en lenguajes de bajo nivel como C y C++, estas bibliotecas ofrecen un buen rendimiento para tareas intensivas.

Acceso al Hardware

Interactúan con el hardware del dispositivo a través de la capa HAL, proporcionando una capa de abstracción que oculta las diferencias entre las implementaciones de hardware de diferentes fabricantes.

Native C/C++ Libraries - Ejemplos:

Bibliotecas nativas esenciales para el desarrollo de aplicaciones Android



1. Skia

Renderizado 2D

Utilizada para el renderizado de gráficos 2D en la interfaz de usuario de Android y en el navegador Chrome.

UI

Gráficos 2D

Chrome



2. OpenGL ES

Gráficos 3D

Proporciona APIs para el renderizado de gráficos 3D, utilizado en juegos y aplicaciones con gráficos avanzados.

3D

Juegos

Gráficos



3. SQLite

Base de Datos

Un motor de base de datos ligero utilizado para el almacenamiento de datos estructurados en las aplicaciones.

BD

Ligero

Estructurado



4. Media Framework (libstagefright)

Multimedia

Soporta la reproducción y grabación de varios formatos de audio y video en Android.

Audio

Video

Media

Importante: Estas bibliotecas son utilizadas por el framework de Android a través del HAL y proporcionan funcionalidades esenciales para el desarrollo de aplicaciones móviles.



Características y funciones fundamentales de las librerías nativas C/C++



1. Funcionalidades Fundamentales

Incluyen bibliotecas para gráficos 2D y 3D (**Skia**, **OpenGL ES**), códecs multimedia (**libvpx**, **libstagefright**), bases de datos (**SQLite**), seguridad (**SSL/TLS**, **OpenSSL**), soporte para lenguajes (**libc**), etc.



2. Rendimiento

Al ser escritas en lenguajes de bajo nivel como C y C++, estas bibliotecas ofrecen un **buen rendimiento** para tareas intensivas y operaciones que requieren alta eficiencia.



3. Acceso al Hardware (a través del HAL)

Interactúan con el hardware del dispositivo a través de la **capa HAL**, permitiendo comunicación eficiente entre el software y los componentes físicos del sistema.



4. Optimización

Diseñadas para operaciones de bajo nivel que requieren **alta velocidad** y **eficiencia de recursos** en el sistema operativo Android.



ANDROID RUNTIME (ART)

Entorno de ejecución para aplicaciones Android

El Android Runtime (ART) es el entorno de ejecución para las aplicaciones escritas en los lenguajes de programación de Android (Java y Kotlin). ART es responsable de la ejecución del bytecode Dalvik (en versiones anteriores de Android) o bytecode propio (a partir de Android 5.0 Lollipop).

Función Principal

ART proporciona el entorno en el que el código de la aplicación (escrito en Java o Kotlin) se traduce y se ejecuta en el hardware del dispositivo, optimizando el rendimiento y la eficiencia de la memoria.



Ejecución de Bytecode

ART toma el bytecode DEX de las aplicaciones y lo ejecuta en el dispositivo



Gestión de Memoria

Recolección de basura eficiente para liberar memoria no utilizada



Características y funciones del entorno de ejecución ART

1 Ejecución de Bytecode

ART toma el bytecode DEX (Dalvik Executable) de las aplicaciones Android y lo ejecuta en el dispositivo.

2 Compilación Ahead-of-Time (AOT) y Just-in-Time (JIT)

ART utiliza una combinación de compilación AOT (donde el bytecode se compila a código máquina nativo cuando se instala la aplicación) y JIT (donde el bytecode se compila durante la ejecución) para optimizar el rendimiento de las aplicaciones. **A partir de Android 7.0 Nougat, ART utiliza un enfoque híbrido con perfiles de compilación guiada.**

3 Recolección de Basura (Garbage Collection)

ART gestiona la memoria de las aplicaciones y realiza la recolección de basura para liberar la memoria no utilizada.

4 Soporte para el Lenguaje Java y Kotlin

ART proporciona las bibliotecas base de Java y Kotlin necesarias para ejecutar las aplicaciones escritas en estos lenguajes.

5 Optimización para Dispositivos Móviles

ART está diseñado para ser eficiente en términos de uso de memoria y batería, características cruciales en dispositivos móviles.



ANDROID RUNTIME (ART)

Ejemplo práctico de ejecución de aplicaciones

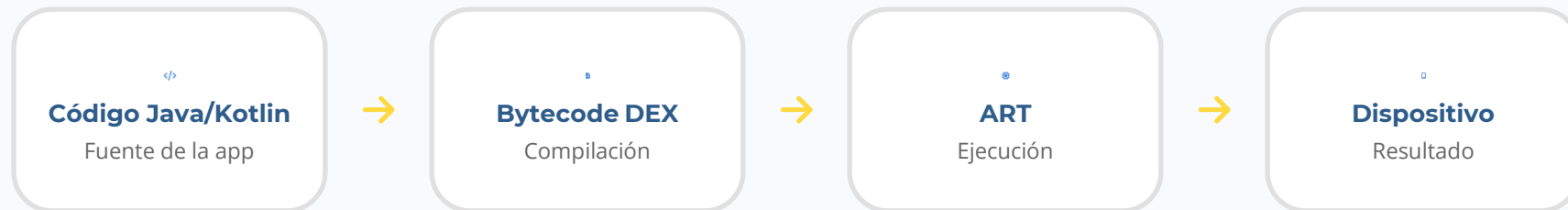


Ejemplo: Cuando se ejecuta una aplicación de Android, el código Java o Kotlin se compila a bytecode DEX, y ART es el encargado de ejecutar este bytecode en el dispositivo.



Proceso de ejecución

El código fuente (Java/Kotlin) se compila a bytecode DEX, que es luego ejecutado por ART mediante la combinación de compilación AOT y JIT para optimizar el rendimiento.





FRAMEWORK DE APLICACIONES (APPLICATION FRAMEWORK)

Componentes y servicios del framework de aplicaciones

El framework de aplicaciones proporciona un conjunto de APIs de alto nivel y bloques de construcción que los desarrolladores utilizan para crear aplicaciones de Android. Simplifica el proceso de desarrollo al proporcionar abstracciones y servicios comunes.



Componentes de la Aplicación

Activities: interfaces de usuario

Services: procesos en segundo plano

Content Providers: gestión de datos compartidos

Broadcast Receivers: receptores de eventos



Gestión de la Interfaz de Usuario

Views, Layouts, Widgets

Herramientas para crear UI interactivas

Soporte para múltiples pantallas

Animaciones y transiciones



Administración de Recursos

Imágenes, archivos de diseño

Cadenas de texto, temas

Recursos de idioma

Assets y archivos raw



Notificaciones e Intents

Mecanismos de notificación

Intents para comunicación entre apps

Broadcasts del sistema

Pending Intents



Componentes y funcionalidades del framework de aplicaciones de Android



1. Componentes de la Aplicación

Define la estructura básica a través de **Activities**, **Services**, **Content Providers** y **Broadcast Receivers**.



2. Gestión de la Interfaz de Usuario

Proporciona herramientas y clases para crear interfaces de usuario interactivas (Views, Layouts, Widgets).



3. Administración de Recursos

Permite acceder a recursos de la aplicación como imágenes, archivos de diseño, cadenas de texto, etc.



4. Administración de Paquetes

Gestiona la instalación, desinstalación y la información de las aplicaciones del sistema.



5. Administración de la Telefonía

Proporciona APIs para acceder a las funcionalidades de telefonía (llamadas, SMS, estado de red).



6. Administración de la Ubicación

Permite acceder a la información de ubicación geográfica y servicios de posicionamiento.



7. Notificaciones

Proporciona mecanismos para mostrar alertas y notificaciones push de estado al usuario.



8. Intents

Un mecanismo para la comunicación asíncrona entre componentes de la aplicación y el sistema.



Framework de Aplicaciones (Application Framework)

Ejemplo: Cuando un desarrollador crea una interfaz de usuario con botones y campos de texto, está utilizando las clases y APIs proporcionadas por el framework de aplicaciones (Views, TextView, Button, LinearLayout, etc.). Cuando una aplicación quiere enviar una notificación, utiliza las APIs del NotificationManager del framework.

```
// Crear un botón programáticamente
Button button = new Button(this);
button.setText("Haz clic");
button.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) {
        // Acción al hacer clic
    }
});

// Enviar notificación
NotificationManager notificationManager =
    (NotificationManager) getSystemService(Context.NOTIFICATION_SERVICE);
notificationManager.notify(id, notification);
```



APIs del Framework

El framework proporciona clases y APIs de alto nivel que simplifican el desarrollo de aplicaciones Android



APLICACIONES (APPLICATIONS)

Capa superior de la arquitectura Android

Esta es la capa superior de la arquitectura de Android y contiene todas las aplicaciones instaladas en el dispositivo, tanto las aplicaciones del sistema (preinstaladas) como las aplicaciones de terceros instaladas por el usuario.

Características principales

Las aplicaciones de Android se construyen utilizando las APIs y los componentes proporcionados por el framework de aplicaciones. Cubren una amplia gama de funcionalidades, desde herramientas de productividad y comunicación hasta juegos y aplicaciones de entretenimiento.



Gmail

Sistema



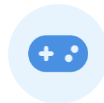
WhatsApp

Terceros



Facebook

Terceros



Juegos

Terceros



APLICACIONES (APPLICATIONS)

Características de las aplicaciones en la capa superior de Android



1. Desarrolladas utilizando el Framework de Aplicaciones

Todas las aplicaciones se construyen utilizando las **APIs y los componentes** proporcionados por el framework de aplicaciones. Esto incluye Activities, Services, Content Providers y Broadcast Receivers.



2. Diversidad de Funcionalidades

Cubren una amplia gama de funcionalidades, desde **herramientas de productividad** y comunicación hasta juegos y aplicaciones de entretenimiento.



3. Ejecutadas en el ART

Se ejecutan dentro del **entorno ART** (Android Runtime), que optimiza el rendimiento y la gestión de memoria de las aplicaciones.



4. Interactúan con las Capas Inferiores

Utilizan las APIs del framework de aplicaciones para acceder a los servicios y recursos proporcionados por las **capas inferiores** (acceso al hardware a través del HAL, almacenamiento a través del kernel, etc.)



APLICACIONES (APPLICATIONS)

Capa superior de la arquitectura Android

Esta es la capa superior de la arquitectura de Android y contiene todas las aplicaciones instaladas en el dispositivo, tanto las aplicaciones del sistema (preinstaladas) como las aplicaciones de terceros instaladas por el usuario.



Gmail
Email



Chrome
Navegador



WhatsApp
Mensajería



Juegos
Entretenimiento



Cámara
Fotografía



Música
Audio



Maps
Navegación



Tienda
Compras



Ejemplos de aplicaciones populares

Gmail, Chrome, WhatsApp, juegos, aplicaciones de redes sociales, etc. Todas estas aplicaciones se construyen utilizando el Framework de Aplicaciones y se ejecutan en el entorno ART.



II. LIBRERÍAS DE ANDROID

Conjuntos de código reutilizable para extender funcionalidad

En el ecosistema de desarrollo de Android, una **librería** se refiere a un conjunto de código (clases, métodos, recursos, etc.) empaquetado de forma reutilizable que los desarrolladores pueden incluir en sus proyectos para extender la funcionalidad de sus aplicaciones sin tener que escribir todo desde cero. Estas librerías pueden ser proporcionadas por Google (dentro del Android SDK), por la comunidad de desarrolladores de código abierto o por empresas de terceros.

Las librerías simplifican tareas comunes como la manipulación de datos, la conexión a redes, la gestión de la interfaz de usuario, el procesamiento de imágenes, la integración con servicios web, y mucho más. Utilizar librerías bien diseñadas no solo acelera el desarrollo, sino que también contribuye a la estabilidad y el rendimiento de las aplicaciones.



SDK Android

Librerías oficiales de Google



Open Source

Comunidad de desarrolladores



Terceros

Empresas especializadas



Reutilizable

Código modular y eficiente



Librerías fundamentales proporcionadas por Google

Estas son las librerías fundamentales proporcionadas por Google como parte del Android Software Development Kit (SDK). Ofrecen la base para construir cualquier aplicación de Android. Estas librerías incluyen componentes esenciales para la gestión del ciclo de vida de las aplicaciones, la interacción con el sistema operativo, la construcción de interfaces de usuario, el acceso a redes, y muchas más funcionalidades básicas que todo desarrollador necesita.

El SDK de Android proporciona un conjunto completo de herramientas y APIs que permiten a los desarrolladores crear aplicaciones de alta calidad. Las librerías están organizadas en paquetes que cubren diferentes aspectos del desarrollo móvil, desde lo más básico hasta funcionalidades avanzadas.



AndroidX

Evolución de la Support Library



Actualizaciones Independientes

Separación del sistema operativo para actualizaciones más frecuentes



Core Android Libraries

Paquetes fundamentales: android.app, android.content, android.os



UI Components

android.view, android.widget para interfaces



Funcionalidades

Servicios del sistema



Redes

android.net para conectividad



Ubicación

android.location para GPS



Gráficos

android.graphics para 2D



Librerías fundamentales proporcionadas por Google



AndroidX

Refactorización de la biblioteca de soporte

AndroidX es una refactorización y mejora de la biblioteca de soporte original de Android. AndroidX separa los componentes del SDK de Android del sistema operativo, lo que permite actualizaciones más frecuentes e independientes.

Ventajas principales:

- ✓ Actualizaciones independientes del sistema operativo
- ✓ Compatibilidad con versiones anteriores
- ✓ Mejoras constantes en rendimiento y seguridad



Recomendación

Preferir AndroidX para actualizaciones independientes del SO



Core Android Libraries

Paquetes fundamentales del SDK

Core Android Libraries incluyen paquetes fundamentales para el desarrollo de aplicaciones Android:

`android.app`

`android.content`

`android.os`

`android.view`

`android.widget`

`android.net`

`android.graphics`

`android.media`

`android.location`

Estos paquetes proporcionan funcionalidades esenciales como gestión del ciclo de vida de las aplicaciones, interacción entre aplicaciones y datos, servicios del sistema operativo, construcción de interfaces de usuario, redes, gráficos 2D, audio, video y servicios de ubicación.



LIBRERÍAS DE SOPORTE DE GOOGLE (FUERA DE ANDROIDX)

Servicios de Google para aplicaciones Android



Firebase SDK

Un conjunto de librerías para acceder a los servicios de Firebase, como autenticación, bases de datos en tiempo real (Realtime Database y Firestore), almacenamiento en la nube (Cloud Storage), funciones sin servidor (Cloud Functions), aprendizaje automático (ML Kit), análisis (Analytics), mensajería en la nube (Cloud Messaging), etc.

✓ Auth

✓ Firestore

✓ Storage

✓ ML Kit

✓ Analytics

✓ FCM



Google Play Services

Un conjunto de APIs y servicios que permiten a las aplicaciones integrar funcionalidades como mapas (Google Maps Android API), publicidad (AdMob), inicio de sesión con Google (Google Sign-In), juegos (Google Play Games Services), etc.

✓ Maps

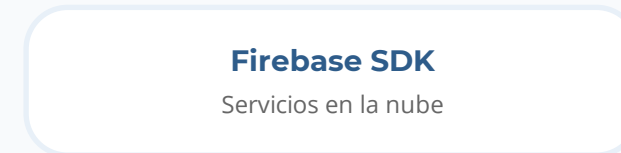
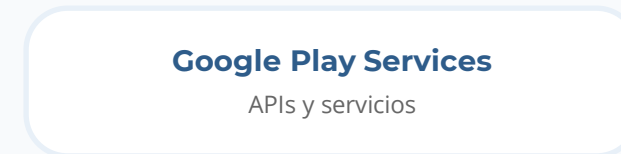
✓ AdMob

✓ Sign-In

✓ Play Games



Arquitectura de Integración





LIBRERÍAS DE TERCEROS Y DE CÓDIGO ABIERTO

Categorías principales de librerías de terceros para Android



Networking

Retrofit, OkHttp para conexiones HTTP



Image Loading and Caching

Glide, Coil para carga de imágenes



Dependency Injection

Hilt, Koin para inyección de dependencias



JSON Parsing

Moshi, Gson para parseo de JSON



Reactive Programming

RxJava, Coroutines para programación reactiva



Logging

Timber para logging eficiente



UI Enhancements

Librerías para mejorar la UI



Testing

JUnit, Espresso para testing



III. ENTORNO DE APLICACIÓN EN ANDROID

Componentes clave del entorno de aplicación

El entorno de aplicación en Android se refiere al conjunto de recursos, servicios y la infraestructura proporcionada por el sistema operativo que permite que una aplicación se ejecute correctamente e interactúe con el usuario y el sistema. Este entorno está cuidadosamente gestionado por Android para garantizar la seguridad, la estabilidad y la eficiencia del dispositivo. Los componentes clave del entorno de aplicación incluyen el proceso de la aplicación, la máquina virtual (ART), la gestión de la memoria, la gestión del ciclo de vida, los permisos y la seguridad.



Importancia del entorno de aplicación

Comprender el entorno de aplicación es fundamental para desarrollar apps eficientes, seguras y con buena experiencia de usuario. Cada componente desempeña un papel crucial en el funcionamiento óptimo de las aplicaciones.



PROCESO DE LA APLICACIÓN (APPLICATION PROCESS)

Aislamiento y gestión de procesos en Android



Proceso de la Aplicación

Cada aplicación de Android se ejecuta en su propio proceso Linux aislado. Este aislamiento es un aspecto fundamental de la seguridad en Android, ya que impide que una aplicación interfiera directamente con los recursos de otras aplicaciones o del sistema operativo.

Aislamiento: Cada aplicación tiene su propio espacio de memoria virtual, lo que significa que no puede acceder directamente a la memoria de otras aplicaciones.

Identificador Único (UID): A cada paquete de aplicación se le asigna un identificador de usuario único (UID) por el sistema al instalarse.

Gestión del Ciclo de Vida: El sistema operativo Android es responsable de iniciar, detener y gestionar el ciclo de vida de los procesos.

Comunicación Inter-Procesos (IPC): Para que las aplicaciones puedan interactuar entre sí, Android proporciona mecanismos de Comunicación Inter-Procesos.



Seguridad del Sistema



Protección de datos privados y del sistema mediante aislamiento de kernel.



Características del Proceso



Memoria aislada por aplicación



UID único por paquete de app



Gestión automática del ciclo de vida



IPC mediante Intents y Binder



Ejemplo Práctico

Cuando se abren múltiples aplicaciones (Gmail, un juego, WhatsApp), cada una se ejecuta en su propio proceso separado. Si una aplicación falla, generalmente no afecta a las demás gracias a este aislamiento.



PROCESO DE LA APLICACIÓN (APPLICATION PROCESS)

Aislamiento y gestión de procesos en Android

Cada aplicación de Android se ejecuta en su propio proceso Linux aislado. Este aislamiento es fundamental para la seguridad y estabilidad del sistema operativo.



1. Aislamiento

Cada aplicación tiene su propio **espacio de memoria virtual**
No puede acceder directamente a la memoria de otras aplicaciones
El aislamiento protege los recursos del sistema
Los fallos quedan contenidos en el proceso



2. Identificador Único (UID)

Se asigna un **UID único** al instalar la aplicación
Todos los procesos de la app se ejecutan con este UID
Los permisos a nivel de sistema se basan en estos UID
Permite control granular de acceso



3. Gestión del Ciclo de Vida por el Sistema

Android controla **inicio, detención y gestión** de procesos
Prioriza recursos según necesidades del usuario
Termina procesos en background si falta memoria
Optimiza el rendimiento del dispositivo



4. Comunicación Inter-Procesos (IPC)

Mecanismos: **Intents, Content Providers, Binder**
Permite compartir datos entre aplicaciones
Iniciar actividades de otras aplicaciones
Comunicación controlada y segura

PROCESO DE LA APLICACIÓN (APPLICATION PROCESS)

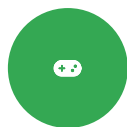
Ejemplo práctico de aislamiento de procesos y comunicación IPC

Ejemplo: Cuando se abren múltiples aplicaciones (Gmail, un juego, WhatsApp), cada una se ejecuta en su propio proceso separado. Si una aplicación falla, generalmente no afecta a las demás gracias a este aislamiento. Cuando una aplicación quiere compartir una foto con otra, utiliza un Intent para solicitar la acción, y el sistema Android media la comunicación.



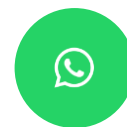
Gmail

Activo



Juego

Activo



WhatsApp

Activo



Cámara

Activo

Seguridad y aislamiento

Cada aplicación tiene su propio espacio de memoria virtual y proceso Linux aislado. La comunicación entre aplicaciones solo es posible a través de mecanismos IPC controlados por el sistema Android, garantizando la seguridad y estabilidad del sistema.

Entorno de ejecución para aplicaciones Android

Como se mencionó en la arquitectura de Android, ART es la máquina virtual que ejecuta el bytecode de las aplicaciones Android. Proporciona el entorno en el que el código de la aplicación (escrito en Java o Kotlin) se traduce y se ejecuta en el hardware del dispositivo.

Función principal de ART

ART (Android Runtime) es responsable de la ejecución del bytecode DEX (Dalvik Executable) de las aplicaciones Android. Proporciona el entorno en el que el código de la aplicación (escrito en Java o Kotlin) se traduce y se ejecuta en el hardware del dispositivo, optimizando el rendimiento y la eficiencia.

Bytecode DEX

Ejecuta el bytecode Dalvik Executable de las aplicaciones

Java y Kotlin

Soporte completo para aplicaciones en Java y Kotlin



Optimización del rendimiento de aplicaciones Android

ART combina compilación AOT y JIT para ofrecer el mejor rendimiento y eficiencia energética

1 Ejecución de Bytecode DEX



ART ejecuta el bytecode Dalvik Executable (.dex) que se encuentra dentro del archivo APK de la aplicación.

2 Compilación AOT y JIT (Híbrida)



ART utiliza una combinación de compilación Ahead-of-Time (AOT) (compilación al instalar) y Just-in-Time (JIT) (compilación durante la ejecución) para optimizar el rendimiento. El enfoque híbrido permite un equilibrio entre el inicio rápido de la aplicación y el rendimiento sostenido.

3 Gestión de Memoria (Garbage Collection)



ART gestiona la asignación y liberación de memoria para las aplicaciones. Utiliza algoritmos de recolección de basura para reclamar la memoria que ya no está en uso, lo que ayuda a prevenir fugas de memoria y a mantener la aplicación responsiva.

4 Seguridad



ART proporciona ciertas características de seguridad al verificar el bytecode y al ejecutar las aplicaciones en un entorno controlado dentro del proceso de la aplicación.

5 Soporte de Bibliotecas Core



ART proporciona acceso a las bibliotecas core de Java y Kotlin necesarias para que las aplicaciones funcionen.

6 Optimización para Dispositivos Móviles



ART está diseñado para ser eficiente en términos de uso de memoria y batería, características cruciales en dispositivos móviles.

Ejemplo práctico de ejecución y gestión de memoria

Ejemplo: Cuando se instala una aplicación desde Google Play Store, el bytecode DEX se compila (parcial o totalmente) a código máquina por ART. Durante la ejecución de la aplicación, ART gestiona la memoria de los objetos creados y destruidos.

Proceso de ejecución

ART toma el bytecode DEX de la aplicación, lo compila a código máquina nativo mediante compilación AOT (Ahead-of-Time) al instalar, y luego gestiona la memoria durante la ejecución con recolección de basura eficiente.

Sistema de gestión de memoria en Android

Android tiene un sistema de gestión de memoria sofisticado diseñado para funcionar eficientemente en dispositivos con recursos limitados (RAM, batería). El sistema operativo debe equilibrar la necesidad de mantener las aplicaciones en ejecución para una experiencia de usuario fluida con la necesidad de liberar memoria para nuevas aplicaciones o procesos del sistema.



Asignación de Memoria

ART gestiona la asignación y liberación de memoria para las aplicaciones

Garbage Collection automática

Liberación de memoria no utilizada

Optimización para dispositivos móviles



Compartición de Memoria

Android intenta compartir memoria entre procesos cuando es posible

Bibliotecas del sistema compartidas

Recursos comunes optimizados

Reducción del uso total de RAM



Liberación de Memoria

Mecanismos para liberar memoria cuando es necesario

Low Memory Killer (LMK)

Cierre de apps en segundo plano

Swapping en algunos dispositivos

Importante

Los desarrolladores deben tener cuidado de liberar referencias a objetos cuando ya no son necesarios para evitar fugas de memoria que puedan llevar a la terminación de su aplicación por el sistema.



Gestión de la Memoria (Memory Management)

Android tiene un sistema de gestión de memoria sofisticado diseñado para funcionar eficientemente en dispositivos con recursos limitados (RAM, batería). El sistema operativo debe equilibrar la necesidad de mantener las aplicaciones en ejecución para una experiencia de usuario fluida con la necesidad de liberar memoria para nuevas aplicaciones o procesos del sistema.

Características:

Asignación y Liberación de Memoria: ART es responsable de asignar memoria a los objetos creados por la aplicación y de liberar esa memoria cuando los objetos ya no son necesarios a través de la recolección de basura.

Compartición de Memoria: Android intenta compartir memoria entre procesos siempre que sea posible (ej., para bibliotecas del sistema o recursos comunes) para reducir el uso total de RAM.

Mecanismos para Liberar Memoria: Cuando la memoria es baja, Android puede recurrir a varias estrategias para liberar memoria:

- **Cerrar aplicaciones en segundo plano:** El sistema prioriza mantener en ejecución las aplicaciones que el usuario está utilizando activamente.
- **Low Memory Killer (LMK):** Un proceso del sistema que identifica y termina procesos menos importantes cuando la memoria es críticamente baja.
- **Swapping:** Algunos dispositivos pueden utilizar una partición de swap, aunque es menos común debido al impacto en el rendimiento.
- **Restricciones de Memoria por Aplicación:** El sistema impone límites; exceder estos límites provoca errores `OutOfMemoryError`.



GESTIÓN DE LA MEMORIA (MEMORY MANAGEMENT)

Ejemplos prácticos de gestión de memoria en Android

Android tiene un sistema de gestión de memoria sofisticado diseñado para funcionar eficientemente en dispositivos con recursos limitados (RAM, batería). El sistema operativo debe equilibrar la necesidad de mantener las aplicaciones en ejecución para una experiencia de usuario fluida con la necesidad de liberar memoria para nuevas aplicaciones o procesos del sistema.

Ejemplos

Si una aplicación se minimiza y no realiza ninguna tarea activa, Android puede optar por terminar su proceso si necesita memoria para una aplicación en primer plano. Los desarrolladores deben tener cuidado de liberar referencias a objetos cuando ya no son necesarios para evitar fugas de memoria que puedan llevar a la terminación de su aplicación por el sistema.

Importante

La gestión de memoria es crucial para el rendimiento y la estabilidad de las aplicaciones Android.

Tip

Utiliza herramientas como Memory Profiler para identificar fugas de memoria.

Application Lifecycle Management

Android gestiona el ciclo de vida de los componentes de una aplicación (Activities, Services, Broadcast Receivers, Content Providers) para asegurar una buena experiencia de usuario y la eficiencia del sistema. El sistema puede iniciar, pausar, detener y destruir estos componentes en función de las interacciones del usuario y el estado del sistema.

Estados de los Componentes

- ✓ onCreate - Inicialización
- ✓ onStart - Visible
- ✓ onResume - Interactiva
- ✓ onPause - Pausada
- ✓ onStop - No visible
- ✓ onDestroy - Destruída

Gestión del Sistema

- Inicio controlado por el sistema
- Pausa por interacción del usuario
- Detención por necesidad de memoria
- Destrucción por el sistema
- Reinicio cuando es necesario

Importante: Los desarrolladores deben implementar correctamente los métodos de callback del ciclo de vida para guardar el estado, liberar recursos y garantizar una experiencia fluida al usuario.



GESTIÓN DEL CICLO DE VIDA DE LAS APLICACIONES (APPLICATION LIFECYCLE MANAGEMENT) - CARACTERÍSTICAS

Características y funciones clave del ciclo de vida de aplicaciones en Android

1 Estados de los Componentes

Cada tipo de componente tiene un ciclo de vida bien definido con estados específicos (ej., **onCreate**, **onStart**, **onResume**, **onPause**, **onStop**, **onDestroy** para una Activity).

2 Callbacks del Ciclo de Vida

El framework de Android proporciona métodos de callback que los desarrolladores pueden sobrescribir en sus componentes para realizar acciones específicas en cada etapa del ciclo de vida (ej., guardar el estado en **onPause**, liberar recursos en **onDestroy**).

3 Priorización de Procesos

Android asigna diferentes niveles de prioridad a los procesos de las aplicaciones en función de los componentes que están activos. Los procesos con componentes visibles para el usuario tienen mayor prioridad y es menos probable que sean terminados.

4 Influencia del Usuario y del Sistema

Las acciones del usuario (abrir una nueva aplicación, volver a una aplicación anterior) y las decisiones del sistema (necesidad de liberar memoria) influyen en las transiciones del ciclo de vida de los componentes.

Ejemplo práctico del ciclo de vida de Activities y Services

Cuando el usuario cambia de una aplicación a otra, la Activity de la primera aplicación pasa por los estados **onPause()** y **onStop()**. Si el sistema necesita memoria, puede destruir la Activity llamando a **onDestroy()**. Cuando el usuario vuelve a la aplicación, la Activity pasa por **onStart()** y **onResume()**.



* Services en segundo plano

Los Services pueden ejecutarse en segundo plano independientemente de las Activities y tienen su propio ciclo de vida (**onCreate**, **onStartCommand**, **onDestroy**). Son ideales para tareas largas como reproducción de música, sincronización de datos o tracking de ubicación.

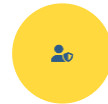
Sistema de permisos y seguridad en Android

El sistema de permisos de Android es un componente clave del entorno de aplicación que controla el acceso de las aplicaciones a recursos sensibles del dispositivo y a ciertas funcionalidades. Esto ayuda a proteger la privacidad del usuario y la integridad del sistema.



Permisos Declarados

Las aplicaciones deben declarar en su archivo AndroidManifest.xml los permisos que necesitan para acceder a recursos protegidos (cámara, ubicación, contactos, internet).



UID y Aislamiento

Cada aplicación tiene un UID único. El aislamiento de procesos basado en UID es una capa fundamental de seguridad.



Permisos en Instalación

Algunos permisos (normales y de firma) se otorgan automáticamente en el momento de la instalación.



Permisos Peligrosos

Los permisos que podrían afectar la privacidad deben ser solicitados en tiempo de ejecución (Android 6.0+).



Seguridad con SELinux

Android utiliza Security-Enhanced Linux (SELinux) para aplicar políticas de control de acceso obligatorio (MAC) a nivel del kernel, proporcionando una capa adicional de seguridad y aislamiento.

PERMISOS Y SEGURIDAD (PERMISSIONS AND SECURITY)

Características del sistema de permisos y seguridad en Android



Permisos Declarados en el Manifiesto

Las aplicaciones deben declarar en su archivo **AndroidManifest.xml** los permisos que necesitan para acceder a recursos protegidos (ej., acceso a la cámara, a la ubicación, a los contactos, a internet).



Permisos en Tiempo de Instalación Normal/Signature



Algunos permisos (normales y de firma) se otorgan automáticamente en el momento de la instalación.



Permisos en Tiempo de Ejecución (Peligrosos)

Los permisos que podrían afectar la **privacidad** o la **seguridad** del usuario (ej., ubicación, cámara, micrófono, contactos) deben ser solicitados y otorgados por el usuario en tiempo de ejecución (a partir de Android 6.0 Marshmallow).



UID y Aislamiento de Procesos

Como se mencionó anteriormente, el **aislamiento de procesos** basado en UID es una capa fundamental de seguridad.



SELinux (Security-Enhanced Linux)

Android utiliza **Security-Enhanced Linux (SELinux)** para aplicar políticas de control de acceso obligatorio (MAC) a nivel del kernel, lo que proporciona una capa adicional de seguridad y aislamiento.

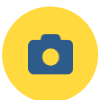


Permisos y Seguridad (Permissions and Security)

Ejemplo: Una aplicación de cámara debe solicitar permiso para acceder a la cámara del dispositivo en tiempo de ejecución. Una aplicación de mensajería necesita permiso para leer los contactos del usuario. El sistema operativo Android utiliza el UID de cada aplicación para controlar su acceso a los archivos y recursos del sistema.

Sistema de Permisos de Android

El sistema de permisos de Android es un componente clave del entorno de aplicación que controla el acceso de las aplicaciones a recursos sensibles del dispositivo y a ciertas funcionalidades. Esto ayuda a proteger la privacidad del usuario y la integridad del sistema.



Permiso de Cámara

Acceso a la cámara del dispositivo para capturar fotos y videos

✓ CAME
RA

✓ RECORD_VID
EO



Permiso de Contactos

Acceso a la lista de contactos del usuario

✓ READ Contac
TS

✓ WRITE Contac
TS



PREGUNTAS FINALES



Pregunta Final

Reflexiona sobre el papel de la HAL en la arquitectura de Android



¿Cuál es la función principal de la capa HAL (Hardware Abstraction Layer) y cómo facilita la portabilidad de Android en diferentes dispositivos?

Analiza cómo la HAL actúa como interfaz entre el framework y el hardware...



Para reflexionar

La HAL define interfaces estándar para que el framework de Android interactúe con el hardware sin conocer los detalles de implementación de cada fabricante.



PREGUNTAS FINALES



Pregunta final

Reflexiona sobre cómo el aislamiento de procesos garantiza la seguridad y estabilidad del sistema



¿Cómo garantiza el "Aislamiento de Procesos" la seguridad y estabilidad del sistema operativo cuando se ejecutan múltiples aplicaciones?

Piensa en cómo Android protege cada aplicación en su propio espacio de memoria...



Para reflexionar

El aislamiento de procesos es fundamental para la seguridad en Android, ya que impide que una aplicación interfiera con los recursos de otras aplicaciones.



PREGUNTAS FINALES



¿Cómo garantiza el Aislamiento de Procesos la seguridad?

¿Cómo garantiza el "Aislamiento de Procesos" la seguridad y estabilidad del sistema operativo cuando se ejecutan múltiples aplicaciones?



Aislamiento de Procesos

Proceso Linux propio por app

UID único por aplicación

Espacio de memoria virtual aislado

Sin acceso directo entre apps



Seguridad

Identificador único (UID)

Aislamiento de memoria

Comunicación solo vía IPC

Protección de datos del sistema



Estabilidad

Fallos contenidos en proceso

No afecta otras apps

Sistema operativo protegido

Recuperación automática

Respuesta Completa



Android asigna a cada aplicación su propio proceso Linux aislado y un **Identificador de Usuario único (UID)**. Esto genera un espacio de memoria virtual propio para cada app, impidiendo que una aplicación acceda o interfiera directamente con los recursos y datos de otra o del sistema operativo. Gracias a este aislamiento, si una aplicación falla, generalmente no afecta el funcionamiento de las demás aplicaciones abiertas ni la estabilidad global del sistema.



PREGUNTAS FINALES



¿Cuál es la función principal de la capa HAL?

¿Cuál es la función principal de la capa HAL (Hardware Abstraction Layer) y cómo facilita la portabilidad de Android en diferentes dispositivos?



Interfaz Estándar

- Actúa como interfaz entre framework y hardware
- Define interfaces abstractas para componentes
- Permite que el framework interactúe sin conocer detalles del fabricante
- Hardware específico controlado por kernel Linux



Abstracción de Hardware

- Oculto diferencias entre implementaciones
- Fabricantes implementan interfaces estándar
- Framework no conoce detalles del hardware
- Compatibilidad garantizada



Módulos HAL

- Implementación a través de módulos (.so)
- Cargados por el sistema según necesidad
- Interfaz estándar para cada tipo de hardware



Facilita Portabilidad

- Android ejecuta en variedad de hardware
- Fabricantes solo proporcionan módulos HAL
- Sin modificar el framework de Android
- Actualizaciones independientes del SO



Framework

Capa superior de Android



HAL

Capa de abstracción



Hardware

Dispositivo físico

✓ CONCLUSIÓN

Resumen de la arquitectura de Android



Arquitectura en Capas

Linux → HAL → Nativas → ART → Framework
→ Apps

Organización modular y eficiente

Separación de responsabilidades clara

Optimización de recursos del sistema



Librerías

AndroidX para componentes modernos

Servicios de Google (Firebase, Play)

Librerías de terceros especializadas

Optimización del desarrollo



Entorno de Aplicación

Procesos aislados y seguros

Gestión de memoria eficiente

Ciclo de vida bien definido

Permisos granulares



Próximo paso

Aplica la separación de responsabilidades, modularidad y buenas prácticas arquitectónicas en tus proyectos móviles. Diseña aplicaciones que sean fáciles de mantener, escalar y testear.